Pokročilé programování (PPRO) · semestrální projekt

Ordinace Na Vyhlídce

Rezervační systém pro recepci ordinace

Technická dokumentace

Zvolené zadání	A – Rezervace v ordinaci
Autor	**Samuel Mader**
Studijní obor	Aplikovaná informatika
Fakulta	Fakulta informatiky a managementu, Univerzita Hradec Králové
Akademický rok	2026/2027, zimní semestr
Repozitář	https://github.com/morenaker/PPRO2026_c
Verze dokumentu	4. 11. 2026

Obsah

1 Úvod ... 3
Pro koho ... 3
Jaký problém řeší ... 3
Stav implementace ... 3
Pravidla, na kterých klient trvá ... 3
2 Datový model ... 4
Vazba M:N ... 4
Entity a klíčové atributy ... 5
Vazby ... 5
Proč exclusion constraint ... 5
3 Architektura ... 6
Průchod požadavku ... 6
Jak je zajištěno, že závislosti vedou jedním směrem ... 6
Připravenost na změnu ... 7
4 Použité technologie ... 7
5 Spuštění ... 8
Prerekvizity ... 8
Postup krok za krokem ... 8
Adresy služeb ... 8
Přihlašovací údaje k testovacím datům ... 8
Zastavení a časté potíže ... 8
6 Uživatelské rozhraní ... 9
7 Testy ... 10
Spuštění testů ... 10
Co je otestované ... 10
8 Rozhodnutí k otevřeným bodům ... 11
9 Seznam změn ... 11
4. 11. 2026 – požadavek na vylepšení UI a dotažení projektu ... 11
10 Protokol o zacyklení ... 13
Chyba 1: „nový vzhled se nenačetl“ ... 13
Chyba 2: „v diáři chybí první rezervace“ ... 13
Opravdové zacyklení: čtení zadání v PDF ... 13
11 Přiznání práce s AI ... 13
Použité nástroje ... 13
Na které části projektu ... 14
Jak byl výsledek ověřen ... 14

> Obsah se aktualizuje ve Wordu: klepněte do něj pravým tlačítkem › Aktualizovat pole.

1 Úvod

Ordinace Na Vyhlídce je webová aplikace pro recepci soukromé ordinace. Nahrazuje papírový diář: eviduje pacienty, lékaře a jejich ordinační hodiny, rezervace na konkrétní termín a k uskutečněné návštěvě provedené výkony z číselníku.

Pro koho

Klientem (podle zadání A) je MUDr. Jana Hrubá a její praxe se třemi lékaři na částečné úvazky a zhruba 900 pacienty. Hlavní uživatelkou je recepční (v zadání paní Věra), která aplikaci obsluhuje při telefonu nebo u přepážky. Proto je rozhraní jednoduché, česky, s málo kroky a rychlým načítáním. Druhou rolí je **správce**, který navíc spravuje ukázková data.

Jaký problém řeší

- **Dvojité objednání.** Na papíře se dvakrát ročně stalo, že přišli dva lidé na stejný čas. Aplikace překryv dvou rezervací u jednoho lékaře nedovolí, a to přímo v databázi.
- **Přehled o dni.** Recepční po přihlášení hned vidí, kdo je dnes objednaný a kdo dnes ordinuje.
- **Ruční přepis výkonů.** Výkony se zapisují k návštěvě z číselníku (kód, název, body), takže měsíční výkaz pro pojišťovnu půjde sestavit z dat.
- **Zrušené termíny.** Zrušení uvolní termín, ale záznam zůstane kvůli statistice opakovaně rušících pacientů.

Stav implementace

Funkce ze zadání	Stav k 4. 11. 2026
Kostra: vrstvy, Docker, migrace, testy, přihlášení, testovací data	hotovo
1 Pacienti – evidence, hledání podle příjmení/telefonu, kontrola duplicity	hotovo
2 Lékaři a ordinační hodiny	zobrazení hotové, zakládání a úprava chybí
Úvod „Dnes“, diář po dnech (čtení)	hotovo
3 Rezervace vč. kontroly ordinačních hodin	model a pravidlo v DB hotové, formulář chybí
4 Záznam návštěvy a výkonů	model hotový, služba a UI chybí
5 Zrušení rezervace a statistika rušících	model hotový, služba a UI chybí
6 Přehled volných termínů (týden × lékař)	chybí
7 Měsíční výkaz výkonů a bodů	chybí

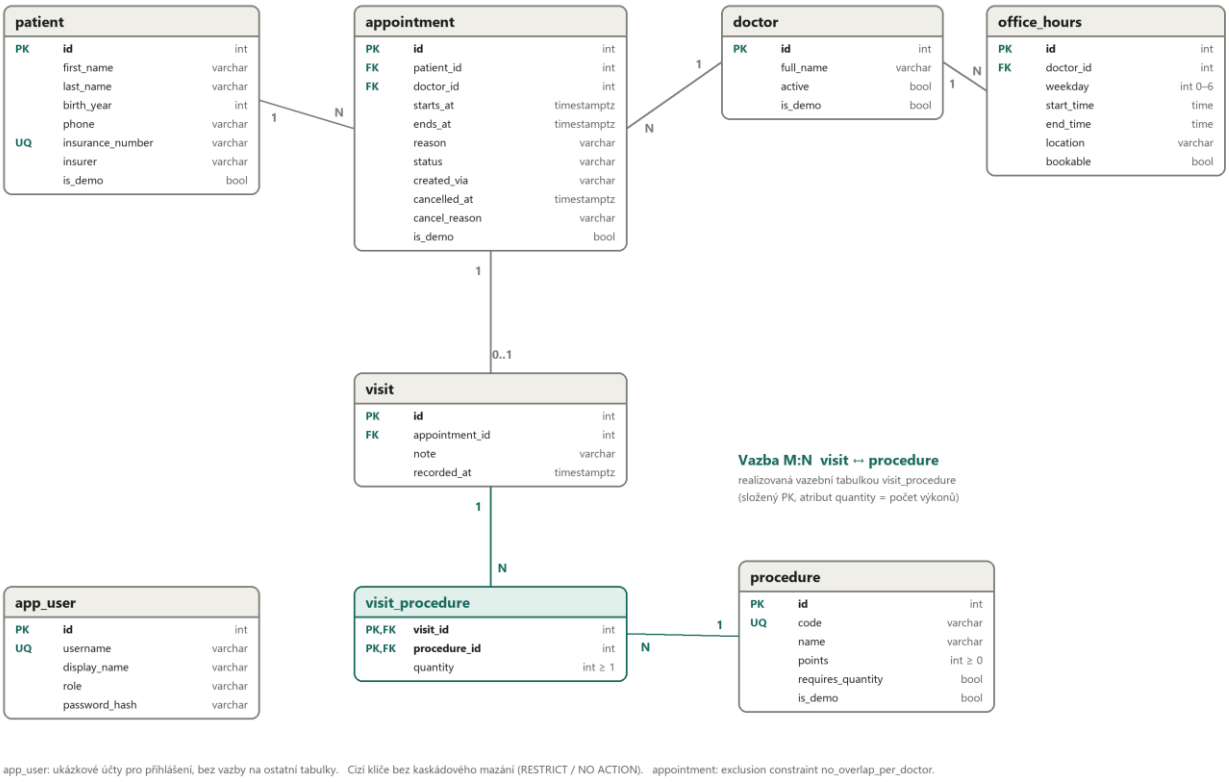
Pravidla, na kterých klient trvá

Pravidlo	Jak se vynucuje
Dvě rezervace na stejný čas u jednoho lékaře nesmí vzniknout	exclusion constraint `no_overlap_per_doctor` v PostgreSQL (rozsah `tstzrange`, jen nezrušené rezervace)
Rezervace jen v ordinačních hodinách, ne na pobočce bez objednávání	příznak `office_hours.bookable`, kontrola ve službě rezervací (plánováno)

Pravidlo	Jak se vynucuje
Výkon jen k uskutečněné návštěvě	`visit` 1:1 k rezervaci se stavem `completed` (pravidlo ve službě plánováno)
Historie pacienta se nemaže	žádné DELETE endpointy, cizí klíče bez kaskádového mazání
Pacient se nesmí založit dvakrát	`patient.insurance_number` UNIQUE + srozumitelná hláška `DuplicatePatientError`

2 Datový model

Schéma je v PostgreSQL 16 a vzniká výhradně verzovanými migracemi Alembic (`alembic/versions/0001_initial_schema.py`, `0002_auth_and_demo_flags.py`). Model má **osm tabulek**, z toho sedm doménových entit a tabulku ukázkových uživatelů.



Obrázek 1: ER diagram. Vazba M:N mezi návštěvou a výkonem je zvýrazněná zeleně.

Vazba M:N

Na jedné návštěvě proběhne několik výkonů a tentýž výkon se opakuje u mnoha návštěv. Vazba **visit ↔ procedure je M:N** a je realizovaná vazební tabulkou `visit_procedure` se složeným primárním klíčem (`visit_id`, `procedure_id`). Tabulka nese navíc atribut `quantity` (počet), protože u některých výkonů (např. aplikace injekce) se zapisuje, kolikrát proběhl.

Entity a klíčové atributy

Tabulka	Účel	Klíčové atributy a omezení
patient	pacient v evidenci	jméno, příjmení, `birth_year`, telefon, pojišťovna; `insurance_number` UNIQUE (podle něj se pozná duplicita)
doctor	lékař	`full_name`, `active`
office_hours	ordinační hodiny lékaře v daný den v týdnu	`weekday` 0–6, `start_time` < `end_time`, `location`, `bookable` (pobočka bez objednávání)
appointment	rezervace termínu	`starts_at`–`ends_at` (timestamptz), `reason`, `status` ∈ {booked, cancelled, completed}, `cancelled_at`, `cancel_reason`, `created_via`; exclusion constraint proti překryvu
visit	záznam uskutečněné návštěvy	`appointment_id` UNIQUE (1:1 k rezervaci), poznámka, čas zápisu
procedure	číselník výkonů	`code` UNIQUE, `name`, `points` ≥ 0, `requires_quantity`
visit_procedure	vazba M:N návštěva ↔ výkon	PK (`visit_id`, `procedure_id`), `quantity` ≥ 1
app_user	ukázkové účty pro přihlášení	`username` UNIQUE, `role` ∈ {admin, reception}, `password_hash` (PBKDF2)

Vazby

- **patient 1 : N appointment** – pacient chodí opakovaně.
- **doctor 1 : N appointment** a **doctor 1 : N office_hours** – každý lékař má jiné dny a časy.
- **appointment 1 : 0..1 visit** – návštěva vzniká jen k rezervaci, která proběhla.
- **visit M : N procedure** přes `visit_procedure`.

Tabulky `patient`, `doctor`, `procedure` a `appointment` mají příznak `is_demo`. Podle něj jde ukázková data odebrat, aniž by se dotkla reálných záznamů (README, rozhodnutí D13).

Proč exclusion constraint

Klient trvá na tom, že dvojí objednání nesmí jít „ani omylem“. Kontrola jen v aplikaci by nepokryla souběh dvou požadavků zároveň. Constraint `EXCLUDE USING gist (doctor_id WITH =, tstzrange(starts_at, ends_at, '[)') WITH &&) WHERE (status <> 'cancelled')` zaručí v databázi, že se rozsahy u stejného lékaře nepřekrývají. Navazující termíny (10:00–10:15 a 10:15–10:30) jsou povolené díky polootevřenému intervalu a zrušená rezervace termín uvolní.

3 Architektura

Aplikace má tři vrstvy a závislosti vedou **jen jedním směrem**:

```
web (FastAPI routery + Jinja2)  →  services (pravidla, transakce)  →  repositories (SQL)
→  models (schéma)
```

Vrstva	Složka	Čím je realizovaná	Smí	Nesmí
Prezentační	app/web/	FastAPI routery, Jinja2 šablony, CSS	parsovat požadavek, zavolat službu, vykreslit šablonu	SQL, obchodní pravidla
Aplikační logika	app/services/	třídy *Service (např. `PatientService`, `OverviewService`)	obchodní pravidla, transakce (`commit`), `DomainError`	znát HTTP/FastAPI, psát SQL
Přístup k datům	app/repositories/	třídy *Repository nad SQLAlchemy 2	dotazy do DB	pravidla, commit
Model	app/models/	deklarativní modely SQLAlchemy + migrace Alembic	schéma a omezení	logiku

Průchod požadavku

Příklad: recepční hledá pacienta „Novák“.

1. Router `GET /pacienti` (`app/web/routers/patients.py`) přečte parametr `q` a zavolá `PatientService(session).search(q)`.
2. Služba ořízne dotaz a předá ho `PatientRepository.search()`.
3. Repository sestaví `SELECT` nad modelem `Patient` a vrátí seznam objektů.
4. Router vykreslí šablonu `patients/list.html`. SQL ani pravidla v routeru nejsou.

Chyba pravidla (např. duplicitní číslo pojištěnce) se ve službě převede na `DomainError` se srozumitelnou českou hláškou a router ji jen zobrazí ve formuláři.

Jak je zajištěno, že závislosti vedou jedním směrem

- **Struktura balíčků.** Každá vrstva má vlastní složku. Služba si repository vytváří sama ze `Session`, router dostane `Session` přes FastAPI `Depends(get_session)` a předá ji službě, sám s ní nepracuje.
- **Automatický test** `tests/test_architecture.py` projde zdrojové soubory každé vrstvy, přečte importy (modul `ast`) a selže, když vrstva importuje něco, co nesmí: router SQLAlchemy výrazy (`select`, `text`) nebo repository, služba FastAPI/Starlette nebo SQL výrazy, repository službu, model cokoli nad sebou. Test byl ověřen tak, že se do routeru schválně vrátil `from sqlalchemy import text` a test spadl.
- **Pravidla pro AI asistenta** v `AGENTS.md` (tabulka „smí / nesmí“) a kontrola rozdílů před commitem.
- **Databázová omezení** jako poslední pojistka: pravidla, na kterých klient trvá, drží databáze i kdyby aplikační kód selhal.

> Při psaní dokumentace (4. 11. 2026) test odhalil dvě skutečná porušení, která byla opravena: endpoint `/health` volal `SELECT 1` přímo v routeru (nyní `HealthService` → `HealthRepository`) a `DemoDataService` sám skládal SQL (nyní `DemoDataRepository`).

Připravenost na změnu

Nová sekce = model (+ migrace) → repository → service → router → položka v `NAV_ITEMS` → šablony → test. Každé rozhodnutí z otevřených bodů zadání je izolované v jedné službě nebo konstantě (README, tabulka Rozhodnutí), aby změna názoru klienta nevedla k přepisu.

4 Použité technologie

Technologie	Verze	Proč byla zvolena
Python	3.12	Čitelný jazyk, ve kterém autor umí kód vysvětlit a na místě změnit.
FastAPI	0.115	Lehký webový framework s typovanými parametry a závislostmi (`Depends`), které přirozeně oddělují vrstvy.
Uvicorn	0.34	Standardní ASGI server pro FastAPI, spouští se jedním příkazem v kontejneru.
Jinja2	3.1	Šablony vykreslované na serveru: rychlé i na slabším tabletu, bez JavaScriptového frameworku.
PostgreSQL	16	Relační DB s exclusion constraintem (`btree_gist`, `tstzrange`), který zaručí zákaz překryvu rezervací.
SQLAlchemy	2.0	ORM s typovanými modely a explicitními dotazy `select()`, vhodné pro vrstvu repository.
Alembic	1.14	Verzované migrace schématu; spouští se automaticky při startu kontejneru.
psycopg	3.2	Moderní ovladač PostgreSQL pro SQLAlchemy.
pydantic-settings	2.7	Konfigurace z proměnných prostředí, žádná hesla v kódu.
itsdangerous	2.2	Podepsaná session cookie pro přihlášení.
pytest + httpx	8.3 / 0.28	Testy služeb proti skutečné DB a HTTP testy přes FastAPI `TestClient`.
Docker Compose	v2	Aplikace i databáze se spustí jedním příkazem bez ruční instalace.
CSS (vlastní)	–	Jediný stylopis s design tokeny (světlý/tmavý režim); žádná těžká knihovna, rychlé načítání.

5 Spuštění

Prerekvizity

- **Docker Desktop** (nebo Docker Engine) s **Docker Compose v2** – příkaz `docker compose version` musí fungovat.
- **Git** pro stažení repozitáře.
- Volný port **8000**. Python ani PostgreSQL se na počítač instalovat nemusí, běží v kontejnerech.

Postup krok za krokem

1. Stáhněte repozitář:

```
git clone https://github.com/morenaker/PPRO2026_c.git
cd PPRO2026_c
```

2. (Volitelné) Pro vlastní hesla zkopírujte `.env.example` na `.env` a upravte hodnoty. Bez toho se použijí vývojové výchozí hodnoty.
3. Spusťte aplikaci. První spuštění stáhne obrazy a sestaví kontejner (několik minut); migrace databáze proběhnou samy:

```
docker compose up --build
```

4. Otevřete v prohlížeči **http://localhost:8000** a přihlaste se ukázkovým účtem (tabulka níže).
5. Pro předvedení s daty se přihlaste jako `admin` a v **Nastavení › Testovací data** klepněte na **Přidat testovací data**. Vznikne 30 pacientů, 3 lékaři s hodinami, 10 výkonů a přibližně 120 rezervací kolem dnešního dne.

Adresy služeb

Služba	Adresa	Poznámka
Webová aplikace	http://localhost:8000	přihlášení, úvod „Dnes“, diář, pacienti, lékaři, nastavení
Kontrola stavu	http://localhost:8000/health	vrací `{"status": "ok"}`, když odpovídá databáze
PostgreSQL	db:5432 (jen uvnitř sítě Dockeru)	databáze `ordinace`, testy používají samostatnou `ordinace_test`

Přihlašovací údaje k testovacím datům

Uživatel	Heslo	Role	Co smí
recepce	recepce-demo	recepce	vše kromě Nastavení
admin	admin-demo	správce	vše + Nastavení (přidání/odebrání testovacích dat)

Jde o vědomě veřejná **ukázková** hesla (README, rozhodnutí D12); data jsou syntetická. Hesla se dají změnit proměnnými `DEMO_RECEPTION_PASSWORD` a `DEMO_ADMIN_PASSWORD`, podpis cookie proměnnou `SECRET_KEY`. Vývojové přihlášení k databázi (`ordinace` / `ordinace_dev`) je výchozí hodnota v `docker-compose.yml` a používá se jen lokálně.

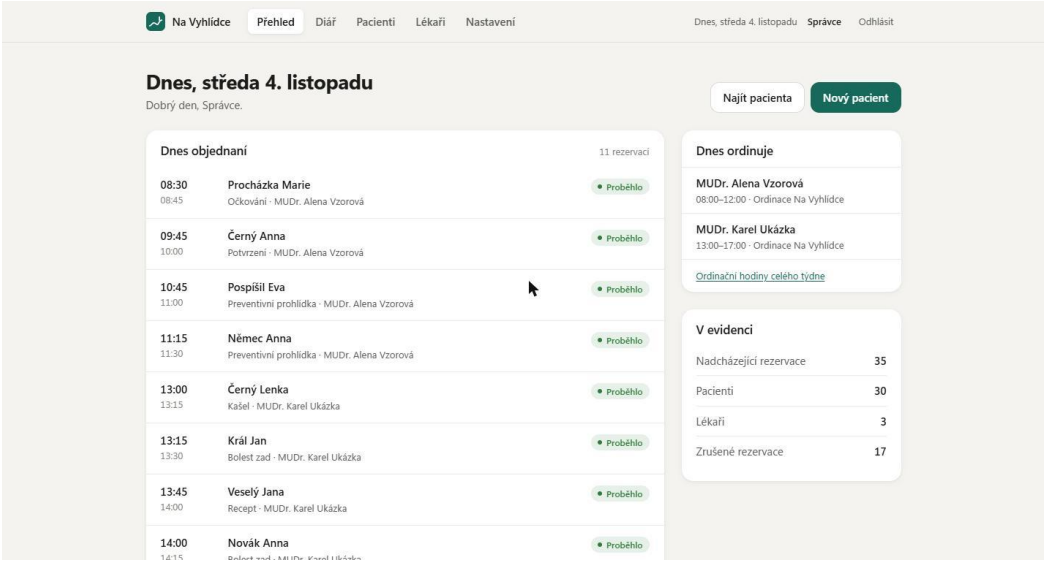
Zastavení a časté potíže

- Zastavení: `Ctrl+C`, nebo `docker compose down`. Úplné smazání dat databáze: `docker compose down -v`.

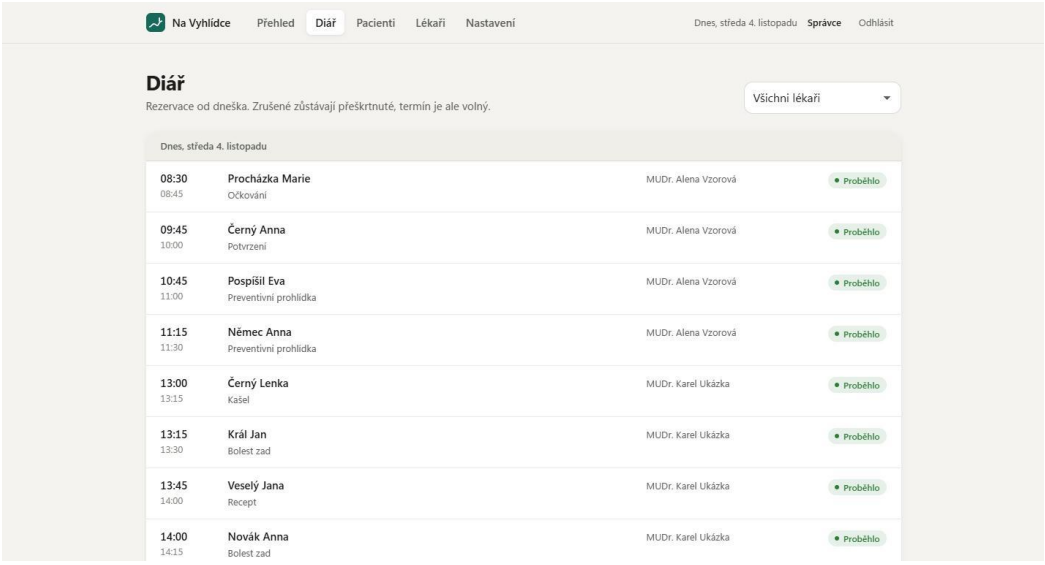
- Port 8000 je obsazený: ukončete jinou aplikaci na tomto portu, nebo v `docker-compose.yml` změňte `"8000:8000"` např. na `"8080:8000"`.
- Po změně kódu v Pythonu: `docker compose restart app` (zdrojové kódy jsou do kontejneru připojené).

6 Uživatelské rozhraní

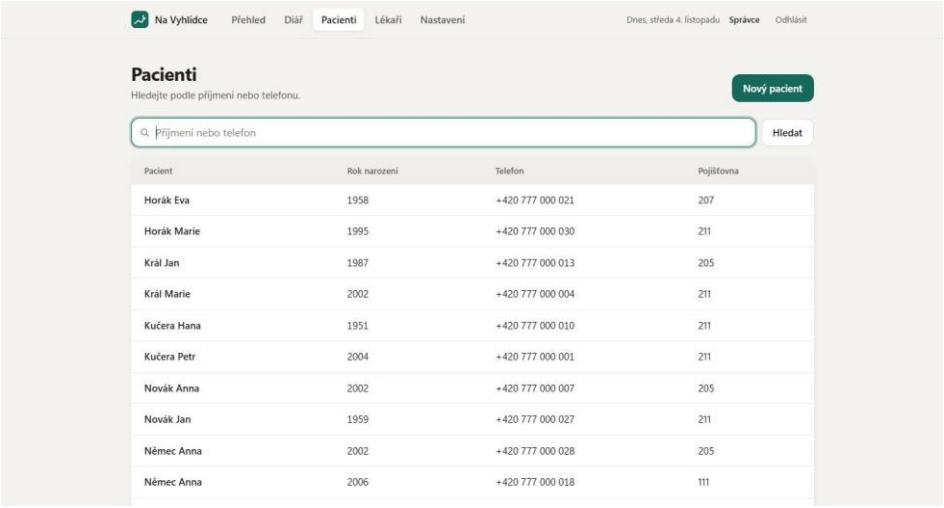
Rozhraní je navržené jako klidný pracovní nástroj pro recepci: systémové písmo, teplé neutrální plochy, jedna akcentní barva, světlý i tmavý režim podle nastavení zařízení a funguje na tabletu (README, rozhodnutí D14).



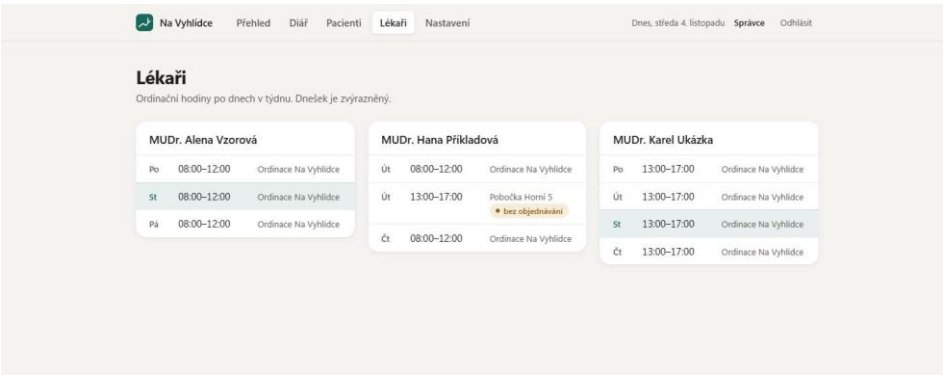
Obrázek 2: Úvod „Dnes“ – dnešní objednaní, kdo dnes ordinuje, souhrn evidence.



Obrázek 3: Diář seskupený po dnech, stav rezervace jako štítek.



Obrázek 4: Evidence pacientů s hledáním podle příjmení nebo telefonu.



Obrázek 5: Ordinační hodiny lékařů, dnešní den je zvýrazněný.

7 Testy

Testy jsou napsané v **pytest**. Běží proti **skutečnému PostgreSQL** (kvůli exclusion constraintu nejde DB nahradit), v samostatné databázi `ordinace_test` vytvořené z migrací. Každý test běží v transakci, která se na konci vrátí zpět, takže testy se navzájem neovlivňují a neovlivní ani vývojová data. HTTP vrstvu testuje FastAPI `TestClient` (httpx).

Spuštění testů

Aplikace musí běžet (`docker compose up`). Pak v druhém terminálu:

```
docker compose exec app pytest              # všechny testy
docker compose exec app pytest -v           # s názvy testů
docker compose exec app pytest tests/test_appointment_overlap.py   # jeden soubor
```

Co je otestované

Soubor	Druh	Co ověřuje
test_auth.py	jednotkový + HTTP	hash a ověření hesla (PBKDF2); nepřihlášený je přesměrován na přihlášení; špatné heslo; přihlášení a odhlášení; recepce nesmí do Nastavení, správce ano
test_patients.py	integrační (DB)	založení pacienta a hledání podle příjmení i telefonu; duplicitní číslo pojištěnce je odmítnuto srozumitelnou chybou; stránky se vykreslí

Soubor	Druh	Co ověřuje
test_appointment_overlap.py	integrační (DB)	databáze odmítne překrývající se rezervace u stejného lékaře; navazující termíny projdou; zrušená rezervace termín uvolní
test_demo_data.py	integrační (DB + HTTP)	přidání testovacích dat, odmítnutí druhého přidání, odebrání maže jen ukázková data a reálná zůstanou, tlačítka v Nastavení
test_pages.py	HTTP	každá obrazovka se vykreslí bez dat i s daty; úvod ukazuje jen dnešní rezervace; prázdné hledání poradí další krok
test_architecture.py	statický	směr závislostí mezi vrstvami (viz kapitola 3)

Stav ke dni odevzdání této verze: **33 testů, všechny prošly** (`33 passed`).

8 Rozhodnutí k otevřeným bodům

Zadání nechává některé body otevřené. Každé rozhodnutí je zapsané v README (tabulka Rozhodnutí) i s důvodem a místem v kódu; zde stručně:

#	Bod	Rozhodnutí
D1	Délka termínu	rezervace má vlastní začátek a konec, výchozí délka 15 minut
D2	Kdo zakládá rezervaci	zatím jen recepce; sloupec `created_via` nechává cestu k online objednávání
D3	Zrušená rezervace	zůstává v DB se stavem `cancelled`, z diáře a volných termínů zmizí
D4	Totožnost pacienta	určuje ji číslo pojištěnce (UNIQUE)
D5	Nemoc lékaře	rezervace se nepřesouvají automaticky, jen se označí k přeobjednání (plánováno)
D6	Ordinace na jiné adrese	`office_hours.bookable = false`, tam se objednávat nedá
D7	Hlídání překryvu	exclusion constraint v PostgreSQL
D8	Mazání	žádné DELETE, cizí klíče bez kaskády
D9	Časová zóna	ukládá se `timestamptz`, zobrazuje `Europe/Prague`
D10	Technologie	FastAPI + Jinja2, PostgreSQL 16, SQLAlchemy 2 + Alembic, pytest, Docker
D11	Návštěva	jen k rezervaci se stavem `completed`, 1:1
D12	Přihlášení	jen ukázkové účty s hashovanými hesly, session v podepsané cookie
D13	Testovací data	příznak `is_demo`, odebrání maže jen je
D14	Vzhled a úvod	klidný pracovní nástroj, úvod „Dnes“ (doplněno 4. 11. 2026)

9 Seznam změn

Úplný seznam je v souboru `docs/ZMENY.md`. Tato kapitola shrnuje poslední požadavek.

4. 11. 2026 – požadavek na vylepšení UI a dotažení projektu

> Požadavek autora: vylepšit vzhled aplikace, aby nepůsobila jako vygenerovaná AI, a doplnit vše ostatní potřebné k odevzdání – technickou dokumentaci, seznam změn a nahrání na GitHub.

Uživatelské rozhraní

- Nový vizuální směr a design tokeny v jednom stylopisu (barvy, zaoblení, stíny, typografická škála), světlý i tmavý režim, přístupnost (fokus, omezení animací).
- Úvod „Dnes“ místo dlaždic s čísly: dnešní objednaní, kdo dnes ordinuje, souhrn evidence.
- Diář seskupený po dnech s lepivou hlavičkou dne a stavovými štítky.
- Pacienti v tabulce, prázdné stavy s radou, co dál; formulář nového pacienta ve dvou sloupcích s nápovědou.
- Lékaři se zvýrazněným dneškem; nové přihlášení s logem; přehlednější Nastavení.
- Responzivita pro tablet a mobil; otisk stylopisu v URL proti zastaralé cache.

Aplikační logika a architektura

- `OverviewService.todays_appointments()` a `AppointmentRepository.list_between()`; český zápis data v šablonách.
- Opraveno porušení vrstev: SQL z endpointu `/health` do `HealthRepository`, SQL z `DemoDataService` do `DemoDataRepository`.

Testy

- Nové `test_architecture.py` (hlídá vrstvy) a `test_pages.py` (vykreslení obrazovek); počet testů 15 → 33.

Dokumentace a repozitář

- Tato dokumentace (`docs/dokumentace.docx` a `.pdf`), ER diagram a snímky v `docs/img/`, seznam změn `docs/ZMENY.md`, rozhodnutí D14 v README.
- Nové skilly pro AI asistenta (vzhled a typografie) a nahrání projektu na GitHub.

10 Protokol o zacyklení

Situace. Při přestavbě vzhledu (4. 11. 2026) agent Claude Code přepsal stylopis a všechny šablony a podle sebe měl hotovo. První kontrola v prohlížeči ale ukázala dvě různé chyby, které vypadaly jako jedna a svedly ke špatné stopě.

Chyba 1: „nový vzhled se nenačetl“

První signál: snímek úvodní stránky vypadal jako holé HTML – odrážky u seznamu rezervací, modré odkazy a místo loga černý čtverec. Zároveň se ale vykreslil nový obsah šablon (nadpis „Dnes, středa 4. listopadu“).

Podle čeho se to poznalo: nové šablony se načetly a nový stylopis ne. Kdyby byla chyba v CSS, rozbila by se jen část pravidel, ne všechna najednou. Černý čtverec byl nové SVG logo bez nových CSS tříd. Prohlížeč tedy použil starou verzi `app.css` z cache, protože adresa souboru se nezměnila.

Jak se to rozseklo: místo dalšího ladění CSS se do odkazu na stylopis přidal otisk obsahu souboru (`app.css?v=<sha1>`). Po restartu se vzhled načetl hned a stejná chyba už nepotká ani recepci po každé změně vzhledu.

Chyba 2: „v diáři chybí první rezervace“

První signál: na úvodní stránce byla první dnešní rezervace 08:30 (Procházka Marie), v diáři ale seznam začínal až 09:45. Nad hlavičkou dne zůstávalo prázdné místo.

Špatná stopa: vypadalo to na chybu v dotazu – diář i úvod filtrují od začátku dne, takže se nabízelo podezření na časovou zónu. Porovnání obou stránek ale ukázalo, že data jsou stejná a liší se jen zobrazení.

Příčina a řešení: hlavička dne měla `position: sticky; top: 56px`, aby při posouvání zůstala pod navigací. Panel kolem ní ale měl `overflow: hidden`, a tím se z něj stal posuvný kontejner. Hlavička se odsadila o 56 px uvnitř panelu a překryla první řádek. Oprava byla jednořádková: `overflow: clip` (ořízne rohy, ale nevytvoří posuvný kontejner). Ověřeno posunutím diáře, hlavičky dnů se správně střídají pod navigací.

Opravdové zacyklení: čtení zadání v PDF

Při čtení PDF se zadáním agent dvakrát po sobě zkusil skoro stejný postup a dostal **stejnou chybu** (`UnicodeEncodeError … cp1250`). Druhý pokus nastavil `PYTHONIOENCODING=utf-8`, jenže Python byl spuštěný v izolovaném režimu `-I`, který proměnné `PYTHON*` ignoruje. Signálem byl identický traceback. Rozseklo to přepnutí kódování přímo ve skriptu (`sys.stdout.reconfigure(encoding="utf-8")`).

> Poučení: jeden snímek obrazovky ani „agent hlásí hotovo“ nestačí. Výsledek se ověřuje porovnáním dvou míst, která mají ukazovat totéž, a když se po změně opakuje stejná chyba, je potřeba změnit předpoklad, ne jen parametr.

11 Přiznání práce s AI

Použité nástroje

Nástroj	K čemu
Claude Code (aplikace Claude, model Claude Opus 5.5, Anthropic)	agent pro psaní kódu, testů, šablon, stylopisu a dokumentace podle pravidel v `AGENTS.md`

Nástroj	K čemu
Skilly v `.claude/skills/`	návody pro agenta: `fastapi`, `python-testing-patterns`, `docker-patterns`, `sqlalchemy-alembic-…`, `frontend-design`, `web-design-guidelines`, `ios-hig-design`, `impeccable`, `emil-design-eng`, `better-typography`, `design-taste-frontend`

Na které části projektu

- Kostra aplikace, datový model a migrace, služby a repository, testy, šablony a stylopis vznikaly s pomocí agenta. Autor zadával požadavky, rozhodoval otevřené body zadání (README, Rozhodnutí) a výsledky kontroloval.
- 4. 11. 2026: přestavba vzhledu (kapitola 9), test architektury, oprava porušení vrstev, ER diagram, snímky obrazovek, tato dokumentace a seznam změn.
- Otázky „co klient chce“ agent nerozhodoval sám; podle `AGENTS.md` patří autorovi. Technická rozhodnutí agent zapisoval do README.

Jak byl výsledek ověřen

- **Testy:** celá sada `pytest` (33 testů) proti skutečné databázi po každé větší změně.
- **Test architektury** ověřený schválně zavedeným porušením (kapitola 3).
- **Ruční kontrola v prohlížeči:** všechny obrazovky ve světlém i tmavém režimu, na šířce tabletu (768 px) a telefonu (375 px), chybová hláška při špatném hesle.
- **Kontrola rozdílů** v kódu před commitem a soulad kódu s README (README je jediný zdroj pravdy).
- Jen syntetická data; v repozitáři nejsou reálná hesla ani osobní údaje.